

```

0000 WITH TEXT_IO; USE TEXT_IO; SEPARATE ( IDL.NAM_PUT )--|-----
--|-----TBL--|-----
EXCEPTION;
FUNCTION UPPER_CASE ( A : STRING ) RETURN STRING IS
    S : STRING(1..A'LENGTH) := A;
    DECAL : CONSTANT := CHARACTER'POS( 'A' ) - CHARACTER'POS(
    'a' );
    BEGIN
        FOR I IN 1..S'LENGTH LOOP
            IF S( I ) IN 'a'..'z' THEN
                S( I ) := CHARACTER'VAL( CHARACTER'POS( S( I ) ) + DECAL );
            END IF;
        END LOOP;
        RETURN S;
    END UPPER_CASE;
--|-----
FUNCTION LOWER_CASE ( A : STRING ) RETURN STRING IS
    S : STRING(1..A'LENGTH) := A;
    DECAL : CO
    NSTANT := CHARACTER'POS( 'a' ) - CHARACTER'POS( 'A' );
    BEGIN
        FOR I IN 1..S'LENGTH LOOP
            IF S( I ) IN 'A'..'Z' THEN
                S( I ) := CH
                ARACTER'VAL( CHARACTER'POS( S( I ) ) + DECAL );
            END IF;
        END LOOP;
        RETURN S;
    END LOWER_CASE;
--|-----
0010 BLE_FILE : TEXT_IO.FILE_TYPE;
    BUFFER : STRING(1..120);
    B_CHAR : CHARACTER;
    B_NUM : INTEGER;
    LAST : NATURAL;
    FIRST_COL,
    LAST_COL : NATURAL;
    LAST_FIELD : FIELD_IDX := 0;
    ATTR_SEEN_FOR_THIS_NODE : BOOLEAN := FALSE;
    PACKAGE MY_INTEGER_IO IS NEW INTEGER_IO(
    INTEGER );
    USE MY_INTEGER_IO;
--|-----
PROCEDURE NIBBLE_NAME IS
    BEGIN
        FIRST_COL := LAST_COL + 1;
        WHILE FIRST_COL <= LAST AND THEN ( BUFFER( FIRST
        _COL ) = ' ' OR ELSE BUFFER( FIRST_COL ) = ASCII.HT ) LOOP
            FIRST_COL := FIRST_COL + 1;
        END LOOP;
        LAST_COL := FIRST_COL;
        LAST_COL <= LAST AND THEN BUFFER( LAST_COL ) /= ' ' AND THEN BUFFER( LAST_COL ) /= ASCII.HT LOOP
            LAST_COL := LAST_COL + 1;
        END LOOP;
        LAST_COL := LAST_COL - 1;
        END NIBBLE_NAME;
    BEGIN
        ATTR_IDX_OF_NODE( 0 ) := 0;
        START_NODE( 0 ) := 0;
        END_NODE( 0 ) := 0;
        TEX
        T_IO.OPEN( TABLE_FILE, TEXT_IO.IN_FILE, NOM_TEXTE & ".tbl" );
        LOOP
            EXIT WHEN END_OF_FILE ( TABLE_FILE );
            GET( TABLE_FILE, B_CHAR );
            IF B_CHAR = 'C' THEN
                GET_LINE( TABLE_FILE, BUFFER, LAST );
                LAST_COL := 0;
                NIBBLE_NAME;
                CLASS_IMAGE( LAST_CLASS )
                := NEW STRING( BUFFER( FIRST_COL..LAST_COL ) );
                START_NODE( LAST_CLASS ) := LAST_NODE;
                LAST_CLASS := LAST_CLASS + 1;
                ELIF B
                CHAR = 'E' THEN
                    GET_LINE( TABLE_FILE, BUFFER, LAST );
                    LAST_COL := 0;
                    NIBBLE_NAME;
                    FOR C IN REVERSE 0..LAST_CLASS-1
                    LOOP
                        IF CLASS_IMAGE( C ).ALL = BUFFER( FIRST_COL..LAST_COL ) THEN
                            END_NODE( C ) := LAST_NODE - 1;
                            END IF;
                        END LOOP;
                        ELIF B_CHAR =
                        'N' THEN
                            GET( TABLE_FILE, B_NUM );
                            GET_LINE( TABLE_FILE, BUFFER, LAST );
                            LAST_COL := 0;
                            IF LAST_NODE /= NODE_IDX
                            ( B_NUM ) THEN
                                SET_OUTPUT( STANDARD_OUTPUT );
                                PUT_LINE( "***** NODES OUT OF SYNC LAST NODE = " & INTEGER'IMAGE( INTEGER( LAST_NODE ) ) & "
                                " & B_NUM & " & NATURAL'IMAGE( INTEGER( B_NUM ) ) );
                                RAISE TBL_ERROR;
                            END IF;
                            NIBBLE_NAME;
                            NODE_IMAGE( LAST_NODE ) := NE
                            W STRING( BUFFER( FIRST_COL..LAST_COL ) );
                            LAST_NODE := LAST_NODE + 1;
                            ATTR_SEEN_FOR_THIS_NODE := FALSE;
                            START_FIELD( LAST_NO
                            DE ) := 1;
                            END_FIELD( LAST_NODE ) := 0;
                            ELIF B_CHAR = 'A' OR B_CHAR = 'B' OR B_CHAR = 'I' THEN
                                GET( TABLE_FILE, B_NUM );
                                LAST_COL := 0;
                                IF B_NUM < 0 THEN
                                    B_NUM := - B_NUM;
                                    NUMERO NEGATIF-ATTR_KIND
                                    ( ATTR_IDX( B_NUM ) ) := 'S';
                                    SEQUENCE/LISTE
                                    ELSE
                                        ATTR_KIND( ATTR_IDX( B_NUM ) ) := B_CHAR;
                                END IF;
                                NIBBLE_NAME;
                                ATTR_IMAGE( ATTR_IDX( B_NUM ) ) := NEW STRING( BUFFER( FIRST_COL..LAST_COL ) );
                                IF LAST_ATTR < ATTR_IDX( B_NUM ) THEN
                                    LAST_ATTR := ATTR
                                    _IDX( B_NUM );
                                END IF;
                                IF NOT ATTR_SEEN_FOR_THIS_NODE THEN
                                    ATTR_SEEN_FOR_THIS_NODE := TRUE;
                                    START_FIELD( LAST_NODE ) := LAST_FIELD;
                                END IF;
                                ATTR_IDX_OF_NODE( LAST_FIELD ) := ATTR_IDX( B_NUM );
                                END_FIELD( LAST_NODE ) := LAST_FIELD;
                                LAST_FIELD := LAST
                                _FIELD + 1;
                                END IF;
                                END LOOP;
                                LAST_NODE := LAST_NODE - 1;
                                LAST_CLASS := LAST_CLASS - 1;
                                TEXT_IO.CLOSE( TABLE_FILE );
                                EXCEP
                                TION WHEN END_ERROR =>
                                TEXT_IO.CLOSE( TABLE_FILE );
                                END READ_TABLES;
                                --|-----
                                END TBL;

```